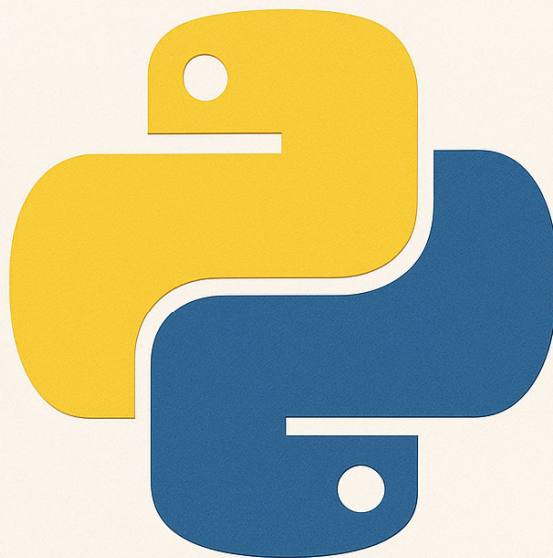


TUTORIAL PYTHON



Luís Simões da Cunha (2025)



Índice

Capítulo 1 – Introdução à Programação com Python 🔄	10
Por que aprender Python? 💡	10
Aplicações académicas e profissionais 🎓👜	10
Instalar Python e escolher o ambiente de trabalho 💻⚙️	10
1. Instalar Python (a linguagem) 🔄	10
2. Escolher um bom editor de código 👤	10
3. Alternativas populares	11
Primeiros passos no terminal e modo interativo 🧪	11
1. Modo interativo	11
2. Criar e executar um ficheiro .py	11
Recado final para este capítulo ✉️	11
Capítulo 2 – O Básico dos Básicos: Variáveis, Tipos e Operações 🗨️	12
Variáveis: Caixas para guardar informação 📦	12
Tipos de dados em Python 🧪	12
Operações Aritméticas $+$ $-$ $\times$ $\div$	13
Operações com texto (strings) 🗑️	13
Booleanos e lógica 🤖	13
Conversões entre tipos 🔄	14
Boas práticas desde o início ✅	14
🧪 Mini-desafio	14
Capítulo 3 – Controlar o Fluxo: Condições e Ciclos 🔄🕒	14
Tomar decisões: if, elif, else 🗨️⚖️	15
Exemplo:	15
Vários elif:	15
Repetições com for 🔄	15
Repetições com while 🔄	16
Palavras mágicas: break, continue, pass ✨	16
Exemplos:	16
Operadores de comparação e lógicos 📊🤖	16
🧪 Mini-desafios	17

Desafio 1 – Condição simples	17
Desafio 2 – Repetição com for	17
Desafio 3 – while com condição.....	17
 Resumo do capítulo	17
Capítulo 4 – Funções: Criar para Reutilizar  	18
O que é uma função? 	18
Exemplo simples:.....	18
Funções com parâmetros 	18
Vários parâmetros 	18
return: devolver um valor 	19
Parâmetros com valor por omissão 	19
*args e **kwargs: parâmetros flexíveis 	19
*args → argumentos posicionais variáveis:.....	19
**kwargs → argumentos nomeados variáveis:	19
Escopo: onde vive a variável? 	19
Boas práticas 	20
 Mini-desafios	20
Desafio 1 – Média de 3 notas	20
Desafio 2 – IMC	20
Desafio 3 – Boas-vindas.....	20
 Resumo do capítulo	20
Capítulo 5 – Estruturas de Dados: Listas, Tuplos, Conjuntos e Dicionários  	21
 Listas: coleções ordenadas e mutáveis 	21
Acessar elementos.....	21
Modificar valores.....	21
Métodos úteis de lista.....	21
Fatiamento (slicing)	21
 Tuplos: listas imutáveis 	22
 Conjuntos: coleções únicas e não ordenadas 	22
Operações com conjuntos.....	22
 Dicionários: pares chave-valor 	22
Acessar e modificar	22
Métodos úteis	22

💡 Quando usar cada uma?	23
🧪 Mini-desafios	23
💡 Resumo do capítulo	23
Capítulo 6 – Manipular Texto como um Pro 📄 ✨	23
Criar e usar strings 📄	23
Métodos úteis de strings 🔧	24
Formatar strings como um mestre 🎨	24
f-strings (Python 3.6+)	24
.format()	24
Alinhamento e casas decimais	24
Juntar e separar texto ✂️	25
Verificações e condições com strings 🔍	25
Expressões regulares (regex) 🧠 🔍	25
🧪 Mini-desafios	25
💡 Resumo do capítulo	26
Capítulo 7 – Datas e Tempos 📅 ⌚	27
O módulo datetime ⌚	27
Criar datas específicas 📅	27
Obter partes de uma data ✂️	27
Calcular diferenças entre datas —	27
Trabalhar com timedelta ⌚	28
Converter string ↔ data 🔄	28
Verificar se é fim de semana 🛌	28
🧪 Mini-desafios	29
💡 Resumo do capítulo	29
Capítulo 8 – Gestão de Ficheiros 📁 📄	30
Abrir ficheiros com open() 📄	30
A forma recomendada: with 🔒	30
Modos de abertura 🔍	30
Escrever em ficheiros 📝	30
Ler linha a linha 📖	31
.readline() vs .readlines()	31

Trabalhar com .csv 📄	31
Escrever CSV:	31
Trabalhar com .json 🔧	31
Caminhos e diretórios 📁	31
🥤 Mini-desafios	32
🧠 Resumo do capítulo	32
Capítulo 9 – Erros, Exceções e Depuração 🐛 🔍	33
Tipos de erros em Python ⚠️	33
❌ Erros de sintaxe (SyntaxError)	33
💥 Erros de execução (RuntimeError)	33
🎯 Erros lógicos	33
Capturar exceções com try / except 🛡️	33
Usar else e finally 🧩	33
Lançar erros com raise 🚨	34
Depuração simples 🕵️	34
Usar print() para inspecionar:	34
Usar breakpoints no VS Code 🔴	34
Boas práticas de tratamento de erros ✅	34
🥤 Mini-desafios	34
🧠 Resumo do capítulo	35
Capítulo 10 – Programação Orientada por Objetos (POO) ⚙️ 🧱	36
O que é um objeto? 🔍	36
Definir uma classe com class ⚖️	36
Explicação:	36
Criar e usar objetos 👤	36
Atributos e métodos ⚙️	37
Atributos	37
Métodos	37
Encapsulamento 🗝️	37
Herança 👤	37
Composição 🧬	37
Métodos especiais (__str__, __repr__) 💎	38
Operadores personalizados 🧠	38

 Mini-projeto	38
 Resumo do capítulo	38
 Revisão rápida: O essencial de uma classe	39
♦ A diferença entre <code>__str__</code> e <code>__repr__</code> 	39
Exemplo:	39
♦ Comparar objetos com operadores (<code>__eq__</code> , <code>__lt__</code> , etc.) 	39
♦ Atributos e métodos de classe (<code>@classmethod</code>) 	40
♦ Métodos estáticos (<code>@staticmethod</code>) 	40
♦ Encapsulamento real com <code>@property</code> 	40
♦ Classe abstrata e polimorfismo 	41
 Boas práticas em POO com Python	41
 Desafios Avançados	41
Capítulo 11 – Introdução a Bibliotecas Científicas  	42
math: funções matemáticas básicas e avançadas  	42
statistics: estatística descritiva 	42
random: geração de números aleatórios 	42
decimal: precisão decimal elevada 	43
fractions: trabalhar com frações reais 	43
 Mini-desafios	43
 Resumo do capítulo	43
Capítulo 12 – NumPy e Computação Numérica  	45
Por que usar NumPy? 	45
Começar com NumPy 	45
Criar arrays (vetores e matrizes) 	45
Operações com arrays    	45
Indexação e slicing 	46
Estatísticas com NumPy 	46
Operações matriciais 	46
Álgebra linear com <code>np.linalg</code> 	46
Comparação com listas 	46
 Mini-desafios	47
 Resumo do capítulo	47

Capítulo 13 – Pandas para Análise de Dados 🐼 📊	48
Começar com Pandas 📦	48
Séries: listas com índice 📈	48
DataFrames: tabelas de dados 📄	48
Acessar colunas e linhas 🔍	48
Filtrar dados 🎯	49
Adicionar e remover colunas 🛠️ ✂️	49
Agrupar e resumir dados 📁	49
Ordenar valores 📄	49
Lidar com dados em falta 💔	49
Importar e exportar ficheiros 📁 📁	49
CSV	49
Excel	49
🧪 Mini-desafios	49
🧠 Resumo do capítulo	50
Capítulo 14 – Visualização de Dados com Matplotlib e Seaborn 📊 🎨	51
Começar com as bibliotecas 📦	51
Gráficos com Matplotlib 📈	51
Gráfico de linhas	51
Gráfico de barras	51
Histograma	51
Seaborn: gráficos bonitos com pouco código 🎨	52
Dispersão (scatterplot)	52
Boxplot (distribuição e outliers)	52
Heatmap (mapa de calor de correlações)	52
Estilo e estética 🎨	52
Guardar o gráfico como imagem 📄	52
🧪 Mini-desafios	52
🧠 Resumo do capítulo	53
Capítulo 15 – APIs e Web Scraping 🌐 🌐	54
O que é uma API? 🤖	54
Usar a biblioteca requests 📄	54
Fazer um pedido GET	54

Exemplo com API pública: exchangerate.host 🌐💱	54
Enviar parâmetros na URL 🗣️	55
Introdução ao Web Scraping 🕸️	55
Exemplo: extrair títulos de uma página 📄	55
Métodos úteis do BeautifulSoup 📦	55
Cuidados e ética no scraping ⚠️	55
🔧 Mini-desafios	56
🧠 Resumo do capítulo	56
Capítulo 16 – Ambientes Virtuais, Módulos e Pacotes 🧪📦	57
Módulos: separar para conquistar 🧩	57
Exemplo:	57
matematica.py:.....	57
programa.py:	57
from ... import ... 🎯	57
Pacotes: pastas de módulos 📁	57
Usar um pacote:.....	57
Instalar bibliotecas com pip 📦	58
Criar o teu próprio pacote 📦	58
Ambientes virtuais com venv 🔒	58
Criar ambiente:	58
Ativar:.....	58
Instalar dentro do ambiente:	58
Desativar:	58
Usar ficheiro requirements.txt 📄	58
Guardar bibliotecas:.....	58
Instalar de um projeto:.....	59
🔧 Mini-desafios	59
🧠 Resumo do capítulo	59
Capítulo 17 – Automação e Tarefas Comuns ⚙️🤖	60
Automatizar renomeações de ficheiros 📁	60
Enviar emails com Python 📧	60
Criar alertas automáticos (ex: preço abaixo de X) 💰	60
Agendar tarefas com schedule ⌚	61

Trabalhar com datas e backups 🕒	61
Limpeza automática de pastas ✂️	61
Monitorizar uma pasta (ex: nova imagem) 📷	61
🧪 Mini-desafios	62
🧠 Resumo do capítulo	62
Capítulo 18 – Projetos Finais para Consolidar 🧪 📊 🧠	63
📊 Projeto 1 – Análise de Dados Económicos (Pandas + Matplotlib).....	63
Funcionalidades:	63
📊 Projeto 2 – Simulador de Carteira de Investimentos (NumPy + POO)	63
Funcionalidades:	63
🌐 Projeto 3 – Ferramenta de Scraping de Dados Públicos (Requests + JSON)	64
Exemplo: previsão meteorológica.....	64
Possíveis variações:	64
🧪 Sugestões de Projetos Alternativos.....	64
🧠 Resumo do capítulo	64

Capítulo 1 – Introdução à Programação com Python

Por que aprender Python?

Vivemos numa era em que programar é uma das competências mais valorizadas — seja na ciência, na economia, na engenharia ou nas humanidades. E se há uma linguagem que se destaca pela simplicidade e poder, é o Python.

Python é como uma caneta mágica: com ela podes desenhar desde um gráfico simples até uma aplicação de inteligência artificial. É uma linguagem que **fala a linguagem dos humanos**, com uma sintaxe limpa, elegante e fácil de ler. E por isso é usada por gigantes como a Google, a NASA, e universidades em todo o mundo.

Aplicações académicas e profissionais

Python é a escolha certa se estás em áreas como:

- **Matemática ou Física:** modelação, simulações, análise de dados;
- **Finanças ou Economia:** cálculos numéricos, estatística, automatização de relatórios;
- **Engenharia ou Computação:** desenvolvimento de software, controlo de sistemas, inteligência artificial;
- **Ciências Sociais ou Humanas:** análise de texto, scraping da web, visualização de dados;
- **Ensino:** é ideal para introduzir alunos à lógica da programação.

E não precisas de ser um programador para usá-la: basta queres **automatizar tarefas**, **analisar dados** ou **testar ideias de forma rápida**.

Instalar Python e escolher o ambiente de trabalho

Antes de escrevermos o nosso primeiro código, vamos preparar o ambiente.

1. Instalar Python (a linguagem)

1. Vai a python.org
2. Escolhe a versão mais recente do Python 3
3. Instala como qualquer outro programa no Windows, macOS ou Linux

 Após a instalação, deves conseguir abrir o terminal e escrever:

```
python --version
```

e ver algo como:

```
Python 3.13.3
```

2. Escolher um bom editor de código

Há muitos, mas recomendamos **Visual Studio Code (VS Code)**:

- Gratuito e multiplataforma
- Intuitivo e leve
- Com suporte a extensões inteligentes como o Copilot

Instalação:

- Vai a code.visualstudio.com
- Instala como qualquer outro programa
- Ao abrires, instala a extensão oficial de Python (ícone dos quadradinhos → “Python” → Instalar)

3. Alternativas populares

- **Jupyter Notebook:** Ideal para ciência de dados e ensino. Cada célula pode conter texto ou código executável.
 - **Anaconda:** Distribuição completa com Python + bibliotecas + Jupyter, ideal para quem quer tudo a funcionar sem complicações.
-

Primeiros passos no terminal e modo interativo

1. Modo interativo

Abre o terminal (ou a consola do VS Code):

```
python
```

Vais ver algo como:

```
Python 3.13.3 (default, ...)
>>>
```

Este >>> é o teu novo amigo: é aqui que podes experimentar comandos rapidamente.

```
>>> 2 + 2
4
>>> print("Olá, mundo!")
Olá, mundo!
```

Para sair, escreve `exit()` ou carrega em `Ctrl + Z` no Windows (`Ctrl + D` no macOS/Linux).

2. Criar e executar um ficheiro `.py`

1. Abre o VS Code
2. Cria um novo ficheiro e chama-lhe `ola.py`
3. Escreve o seguinte:

```
print("Olá, Python!")
```

4. Carrega em `F5` ou clica com o botão direito e escolhe “**Run Python File in Terminal**”

Boom ! O teu primeiro programa está no mundo.

Recado final para este capítulo

Neste capítulo, preparaste o ambiente e deste os teus primeiros passos com Python. A partir daqui, vais entrar num mundo onde vais **controlar o computador com ideias**, transformar dados em decisões e resolver problemas como um verdadeiro explorador digital.

Capítulo 2 – O Básico dos Básicos: Variáveis, Tipos e Operações



Neste capítulo, vais aprender os ingredientes fundamentais para escrever código em Python: **variáveis**, **tipos de dados** e **operações básicas**. Com estes elementos, já poderás construir mini-programas úteis e experimentar com a tua criatividade!

Variáveis: Caixas para guardar informação

Uma **variável** em Python é como uma caixinha com um nome, onde guardas um valor que podes reutilizar ou modificar.

```
nome = "Ana"
idade = 25
altura = 1.68
```

- nome guarda uma string (texto)
- idade guarda um número inteiro (int)
- altura guarda um número com vírgula (float)

Podes alterar o conteúdo a qualquer momento:

```
idade = idade + 1 # agora vale 26
```

Dica: O nome da variável deve ser claro e não pode começar por número nem ter espaços. Usa `_` para separar palavras (`preco_total`, `data_nascimento`).

Tipos de dados em Python

Python identifica automaticamente o tipo de dado de cada variável.

Tipo	Exemplo	Nome técnico
Texto	"Olá"	str
Inteiro	42	int
Decimal	3.14	float
Lógico	True ou False	bool

Usa `type()` para descobrir o tipo de uma variável:

```
type(nome)    # <class 'str'>
type(idade)   # <class 'int'>
```

Operações Aritméticas $+$ $-$ $\times$ $\div$

Python permite fazer contas com operadores simples:

```
a = 10
b = 3
```

Operador	Descrição	Exemplo	Resultado
+	Soma	a + b	13
-	Subtração	a - b	7
*	Multiplificação	a * b	30
/	Divisão (float)	a / b	3.33...
//	Divisão inteira	a // b	3
%	Resto da divisão	a % b	1
**	Potência	a ** b	1000

Operações com texto (strings)

Python trata texto como algo com o qual também se pode "brincar":

```
nome = "Ana"
sobrenome = "Silva"
nome_completo = nome + " " + sobrenome
print(nome_completo) # Ana Silva
```

 Multiplicar strings:

```
print("Olá! " * 3) # Olá! Olá! Olá!
```

 Obter o tamanho do texto:

```
len(nome_completo) # número de caracteres
```

Booleanos e lógica

Variáveis booleanas guardam **valores de verdade**:

```
estudante = True
tem_bolsa = False
```

 Podes combinar condições com operadores lógicos:

Operador	Significado	Exemplo
==	Igual a	x == 10
!=	Diferente de	x != 5
<, >	Menor, Maior	idade > 18

Operador	Significado	Exemplo
<=, >=	Menor ou igual, etc	altura >= 1.6
and	E	True and False
or	Ou	True or False
not	Não (negação)	not True → False

Conversões entre tipos

Às vezes, precisas de transformar um tipo noutro:

```
idade = 25
idade_str = str(idade)      # "25"
altura = float("1.75")     # 1.75
numero = int("42")         # 42
```

⚠ Se tentares converter "vinte" para int, vai dar erro.

Boas práticas desde o início

- Usa nomes descritivos (preco_unitario, não x)
- Escreve código limpo, com espaços e alinhado
- Comenta o que não for óbvio:

```
# Calcular o IMC (Índice de Massa Corporal)
imc = peso / (altura ** 2)
```

Mini-desafio

Escreve um programa que:

1. Guarda o nome, a idade e a cidade onde vives
2. Imprime algo como:

Olá, sou a Marta, tenho 22 anos e vivo em Coimbra.

Depois tenta mudar os valores e ver o resultado!

Capítulo 3 – Controlar o Fluxo: Condições e Ciclos

Chegou a hora de tornar os programas mais **inteligentes e dinâmicos**! Neste capítulo vais aprender a tomar decisões e a repetir ações com os famosos if, for e while.

Tomar decisões: if, elif, else

Muitas vezes queremos que o programa **só execute algo se** uma condição for verdadeira. Para isso usamos:

```
if condição:
    # faz algo
elif outra_condição:
    # faz outra coisa
else:
    # faz algo diferente
```

Exemplo:

```
idade = 17
```

```
if idade >= 18:
    print("És maior de idade.")
else:
    print("És menor de idade.")
```

Vários elif:

```
nota = 15
```

```
if nota >= 18:
    print("Excelente!")
elif nota >= 14:
    print("Bom.")
elif nota >= 10:
    print("Suficiente.")
else:
    print("Reprovado.")
```

💡 A indentação (espaço à frente de cada bloco) é **obrigatória** em Python!

Repetições com for

Quando queremos **fazer algo várias vezes**, usamos o ciclo for.

```
for i in range(5):
    print(i)
```

● Saída:

```
0
1
2
3
4
```

👉 range(5) gera os números de 0 a 4. Se quiseres começar noutro número:

```
for i in range(1, 6):  
    print(i) # de 1 a 5
```

📦 Podes também percorrer listas:

```
nomes = ["Ana", "Bruno", "Carla"]
```

```
for nome in nomes:  
    print("Olá", nome)
```

Repetições com `while`

O `while` repete **enquanto uma condição for verdadeira**.

```
contador = 0
```

```
while contador < 3:  
    print("Contador:", contador)  
    contador += 1
```

⚠ Cuidado com ciclos infinitos! Se a condição nunca for falsa, o ciclo nunca acaba.

Palavras mágicas: `break`, `continue`, `pass` ✨

- `break` → **sai do ciclo**
- `continue` → **salta o resto do ciclo e volta ao início**
- `pass` → **não faz nada (útil como placeholder)**

Exemplos:

```
for i in range(5):  
    if i == 3:  
        break  
    print(i) # imprime 0, 1, 2
```

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i) # imprime 0, 1, 3, 4
```

```
for i in range(5):  
    pass # ainda vamos decidir o que fazer aqui
```

Operadores de comparação e lógicos

Operador	Significado
<code>==</code>	igual

Operador	Significado
!=	diferente
> <	maior, menor
>= <=	maior/menor ou igual
and	e
or	ou
not	não (negação)

💬 Exemplo:

```
idade = 20
tem_carta = True

if idade >= 18 and tem_carta:
    print("Podes conduzir.")
```

Mini-desafios

Desafio 1 – Condição simples

Pede ao utilizador a idade com `input()` e diz se pode ou não votar.

Desafio 2 – Repetição com `for`

Imprime os 10 primeiros números pares.

Desafio 3 – `while` com condição

Pede uma palavra ao utilizador até ele escrever "sair".

Resumo do capítulo

- Usa `if/elif/else` para **tomar decisões**
- Usa `for` para **repetições com número conhecido**
- Usa `while` para **repetições com condição**
- Usa `break`, `continue` e `pass` para controlar o ciclo

Capítulo 4 – Funções: Criar para Reutilizar

Já aprendeste a criar variáveis, usar condições e repetir instruções. Mas e se precisasses de repetir um conjunto de instruções várias vezes em partes diferentes do teu programa? É aqui que entram as **funções**! 💡

O que é uma função?

Uma função é **um bloco de código reutilizável** com um nome. Podes "chamar" essa função sempre que precisares que ela execute determinada tarefa.

Exemplo simples:

```
def saudacao():  
    print("Olá! Bem-vindo ao meu programa.")
```

Para usar esta função, basta chamá-la:

```
saudacao()
```

 Podes chamar a função **quantas vezes quiseres**!

Funções com parâmetros

As funções podem receber **valores de entrada** chamados **parâmetros**:

```
def apresentar(nome):  
    print("Olá,", nome)
```

Chamadas possíveis:

```
apresentar("Marta")  
apresentar("Tiago")
```

 Cada chamada envia um argumento (ex: "Marta") para o parâmetro nome.

Vários parâmetros

```
def somar(a, b):  
    resultado = a + b  
    print("Soma:", resultado)
```

Chamada:

```
somar(3, 5) # imprime "Soma: 8"
```

return: devolver um valor

Funções podem devolver valores com return.

```
def quadrado(x):  
    return x * x
```

Exemplo de uso:

```
y = quadrado(4)  
print(y) # 16
```

👉 Ao usar return, a função devolve o resultado que pode ser guardado numa variável.

Parâmetros com valor por omissão

```
def cumprimentar(nome="amigo"):  
    print("Olá,", nome)
```

Chamadas possíveis:

```
cumprimentar("João") # Olá, João  
cumprimentar()       # Olá, amigo
```

*args e **kwargs: parâmetros flexíveis

*args → argumentos posicionais variáveis:

```
def somar_tudo(*numeros):  
    return sum(numeros)
```

```
print(somar_tudo(1, 2, 3, 4)) # 10
```

**kwargs → argumentos nomeados variáveis:

```
def apresentar_dados(**dados):  
    for chave, valor in dados.items():  
        print(chave, "=", valor)
```

```
apresentar_dados(nome="Ana", idade=30, cidade="Lisboa")
```

Escopo: onde vive a variável?

- Variáveis criadas dentro da função **existem só ali** (escopo local)
- Variáveis criadas fora da função **existem no programa todo** (escopo global)

```
mensagem = "Olá!"
```

```
def mostrar():  
    mensagem = "Olá dentro da função"
```

```
print(mensagem)

mostrar()      # dentro
print(mensagem) # fora
```

Boas práticas

- Usa nomes claros para funções e parâmetros (`calcular_media`, `nome_aluno`)
 - Uma função → uma responsabilidade
 - Usa comentários e docstrings:
- ```
def calcular_area(base, altura):
 """Calcula a área de um triângulo."""
 return (base * altura) / 2
```
- 

## Mini-desafios

### Desafio 1 – Média de 3 notas

Cria uma função que receba 3 notas e devolva a média.

### Desafio 2 – IMC

Cria uma função que receba peso e altura, devolva o IMC e diga se está saudável.

### Desafio 3 – Boas-vindas

Cria uma função com parâmetro opcional `nome="convidado"` que cumprimente a pessoa.

---

## Resumo do capítulo

- Funções evitam repetição e tornam o código mais limpo
- Podes passar valores (parâmetros) e obter resultados com `return`
- Usa `*args` e `**kwargs` quando quiseres mais flexibilidade
- Aprende a distinguir variáveis locais e globais

# Capítulo 5 – Estruturas de Dados: Listas, Tuplos, Conjuntos e Dicionários

Em programação, organizar dados de forma eficiente é essencial. Python oferece quatro estruturas de dados integradas muito poderosas:

- **Listas**  – coleções ordenadas e mutáveis
- **Tuplos**  – coleções ordenadas e imutáveis
- **Conjuntos**  – coleções não ordenadas e sem duplicados
- **Dicionários**  – pares chave-valor

Neste capítulo vais aprender a criar, manipular e aplicar estas estruturas no teu código!

---

## Listas: coleções ordenadas e mutáveis

Uma **lista** guarda vários valores numa ordem definida.

```
frutas = ["maçã", "banana", "laranja"]
```

### Acessar elementos

```
print(frutas[0]) # "maçã"
print(frutas[-1]) # "laranja" (último)
```

### Modificar valores

```
frutas[1] = "manga"
```

### Métodos úteis de lista

| Método                   | Efeito                             |
|--------------------------|------------------------------------|
| <code>append(x)</code>   | adiciona x ao fim da lista         |
| <code>insert(i,x)</code> | insere x na posição i              |
| <code>remove(x)</code>   | remove a 1.ª ocorrência de x       |
| <code>pop()</code>       | remove e devolve o último elemento |
| <code>sort()</code>      | ordena a lista                     |
| <code>reverse()</code>   | inverte a ordem                    |
| <code>copy()</code>      | cria uma cópia da lista            |
| <code>clear()</code>     | esvazia a lista                    |

### Fatiamento (slicing)

```
pares = [2, 4, 6, 8, 10]
print(pares[1:4]) # [4, 6, 8]
print(pares[:2]) # [2, 4]
```

## Tuplos: listas imutáveis

Semelhante às listas, mas **não se podem alterar** após criadas.

```
coordenadas = (10, 20)
```

✓ Úteis quando não queres que os dados sejam modificados (ex: datas, posições fixas).

---

## Conjuntos: coleções únicas e não ordenadas

Um **conjunto** (set) é ótimo para eliminar duplicados.

```
cores = {"vermelho", "verde", "azul"}
```

```
cores.add("amarelo")
cores.remove("verde")
```

### Operações com conjuntos

```
A = {1, 2, 3}
B = {3, 4, 5}
```

```
print(A | B) # união → {1,2,3,4,5}
print(A & B) # interseção → {3}
print(A - B) # diferença → {1,2}
print(A ^ B) # simétrico → {1,2,4,5}
```

⚠ Conjuntos **não preservam ordem** e **não permitem elementos repetidos**.

---

## Dicionários: pares chave-valor

Uma das estruturas mais poderosas de Python!

```
aluno = {
 "nome": "João",
 "idade": 20,
 "curso": "Matemática"
}
```

### Acessar e modificar

```
print(aluno["nome"])
aluno["idade"] = 21
aluno["nota_final"] = 17.5
```

### Métodos úteis

| Método   | Efeito                   |
|----------|--------------------------|
| keys()   | devolve todas as chaves  |
| values() | devolve todos os valores |

| Método                    | Efeito                               |
|---------------------------|--------------------------------------|
| <code>items()</code>      | pares chave-valor                    |
| <code>get("chave")</code> | devolve valor ou None se não existir |
| <code>pop("chave")</code> | remove a chave                       |
| <code>clear()</code>      | esvazia o dicionário                 |
| <code>copy()</code>       | cópia independente                   |

## Quando usar cada uma?

| Estrutura  | Características           | Exemplo de uso                        |
|------------|---------------------------|---------------------------------------|
| Lista      | Ordenada, mutável         | lista de compras                      |
| Tuplo      | Ordenado, imutável        | coordenadas, datas fixas              |
| Conjunto   | Sem ordem, sem repetições | etiquetas únicas, eliminar duplicados |
| Dicionário | Pares chave-valor, rápido | registo de alunos, produtos, etc.     |

## Mini-desafios

1. **Lista de tarefas:** cria uma lista, adiciona tarefas, remove uma e mostra todas.
2. **Tuplo de temperaturas:** cria um tuplo com as temperaturas da semana e mostra a mais alta.
3. **Conjunto de nomes:** remove duplicados de uma lista com nomes repetidos.
4. **Dicionário de produto:** guarda nome, preço e stock; imprime mensagem formatada.

## Resumo do capítulo

- **Listas:** mutáveis, ordenadas.
- **Tuplos:** imutáveis, ordenadas.
- **Conjuntos:** únicos, sem ordem.
- **Dicionários:** associações rápidas chave → valor.

# Capítulo 6 – Manipular Texto como um Pro

Em Python, as **strings** (cadeias de texto) são tratadas como um tipo especial de objeto com muitos **métodos úteis**. Neste capítulo, vais aprender a trabalhar texto com facilidade, desde manipulações básicas até formatações avançadas e expressões regulares.

## Criar e usar strings

```
mensagem = "Olá, mundo!"
nome = 'Maria'
```

Podes usar aspas simples ou duplas. Para texto com várias linhas:

```
texto = """Isto é
uma string
multilinha."""
```

---

## Métodos úteis de strings 🔧

Python oferece **métodos internos** para manipular texto:

```
frase = " Olá, Python! "
```

| Método                         | Resultado                                |
|--------------------------------|------------------------------------------|
| <code>lower()</code>           | 'olá, python!'                           |
| <code>upper()</code>           | 'OLÁ, PYTHON!'                           |
| <code>strip()</code>           | remove espaços das extremidades          |
| <code>replace("a", "x")</code> | substitui 'a' por 'x'                    |
| <code>split()</code>           | divide em lista pelas palavras           |
| <code>startswith("O")</code>   | True se começar por "O"                  |
| <code>endswith("!")</code>     | True se terminar com "!"                 |
| <code>find("Python")</code>    | índice da palavra (ou -1 se não existir) |
| <code>count("o")</code>        | quantas vezes aparece a letra "o"        |

🧪 Exemplo:

```
frase.strip().lower().replace("olá", "hello")
```

---

## Formatar strings como um mestre 🧠

### f-strings (Python 3.6+)

```
nome = "Rui"
idade = 30
```

```
print(f"{nome} tem {idade} anos.")
```

```
.format()
```

```
print("{} tem {} anos.".format(nome, idade))
```

### Alinhamento e casas decimais

```
valor = 12.3456
print(f"€ {valor:.2f}") # duas casas decimais
print(f"|{nome:^10}|") # centrado
print(f"|{nome:<10}|") # alinhado à esquerda
```



## Juntar e separar texto

```
nomes = ["Ana", "Bruno", "Carla"]
print(", ".join(nomes)) # "Ana, Bruno, Carla"

texto = "Python é incrível"
palavras = texto.split() # ['Python', 'é', 'incrível']
```

---

## Verificações e condições com strings

```
email = "exemplo@dominio.com"
print("@" in email) # True
print(email.endswith(".com")) # True
```

---

## Expressões regulares (regex)

Expressões regulares permitem encontrar padrões complexos. Usa-se o módulo `re`:

```
import re

texto = "O meu número é 912345678"
padrao = r"\d{9}"

resultado = re.search(padrao, texto)
print(resultado.group()) # 912345678
```

| Símbolo | Significado             |
|---------|-------------------------|
| .       | qualquer caractere      |
| \d      | um dígito (0–9)         |
| \w      | letra, número ou " _ "  |
| +       | uma ou mais repetições  |
| *       | zero ou mais            |
| {n}     | exatamente n repetições |

---

## Mini-desafios

- Recebe uma frase do utilizador e imprime-a:
  - sem espaços
  - em maiúsculas
  - invertida
- Verifica se um email termina em `.pt`.
- Usa regex para verificar se um NIF é válido (`\d{9}`).
- Usa f-strings para formatar e imprimir uma fatura com descrição, quantidade, preço e total.

---

## Resumo do capítulo

- Strings têm **métodos internos** poderosos (`strip()`, `replace()`, etc.)
- Usa f-strings para **formatar texto de forma clara**
- Expressões regulares permitem **verificar e extrair padrões complexos**

## Capítulo 7 – Datas e Tempos

Manipular datas e tempos é uma tarefa comum em programação: calcular idades, prazos, contagens decrescentes, diferenças entre datas, etc. Felizmente, o Python tem tudo isso bem resolvido com o módulo `datetime`.

---

### O módulo `datetime`

```
import datetime
```

Cria um objeto `datetime` com data e hora completas:

```
agora = datetime.datetime.now()
print(agora) # ex: 2025-05-15 09:32:10.123456
```

Para data apenas:

```
hoje = datetime.date.today()
print(hoje) # ex: 2025-05-15
```

---

### Criar datas específicas

```
from datetime import date
```

```
evento = date(2026, 1, 1) # ano, mês, dia
print(evento)
```

---

### Obter partes de uma data

```
print(evento.year) # 2026
print(evento.month) # 1
print(evento.day) # 1
```

---

### Calcular diferenças entre datas —

```
data1 = date(2025, 5, 15)
data2 = date(2026, 1, 1)

diferenca = data2 - data1
print(diferenca.days) # número de dias
```

---

## Trabalhar com `timedelta` 🕒

```
from datetime import timedelta
```

```
amanha = hoje + timedelta(days=1)
semana_passada = hoje - timedelta(weeks=1)
```

Também funciona com horas, minutos e segundos:

```
tempo = datetime.datetime.now()
futuro = tempo + timedelta(hours=2, minutes=30)
```

---

## Converter string ↔ data 🔄

```
from datetime import datetime
```

```
texto = "2025-12-25 14:30"
formato = "%Y-%m-%d %H:%M"
```

```
data_hora = datetime.strptime(texto, formato)
print(data_hora)
```

Para o caminho inverso (data → string):

```
print(data_hora.strftime("%d/%m/%Y às %H:%M"))
```

🔑 Principais códigos de formatação:

| Código | Significado     | Exemplo |
|--------|-----------------|---------|
| %Y     | Ano (4 dígitos) | 2025    |
| %m     | Mês (2 dígitos) | 05      |
| %d     | Dia             | 15      |
| %H     | Hora (24h)      | 14      |
| %M     | Minutos         | 30      |
| %S     | Segundos        | 00      |

---

## Verificar se é fim de semana 🛌

```
if hoje.weekday() >= 5:
 print("É fim de semana!")
else:
 print("É dia de semana.")
```

➡ `.weekday()` devolve 0 (segunda) até 6 (domingo)

---

## Mini-desafios

1. Calcula quantos dias faltam para o fim do ano.
  2. Cria uma função que receba uma data de nascimento e diga a idade atual.
  3. Pede ao utilizador uma data e converte-a para o formato “15 de Maio de 2025”.
  4. Mostra a data e hora atual formatada como: 15/05/2025 09:30.
- 

## Resumo do capítulo

- O módulo `datetime` permite trabalhar com **datas, horas e diferenças**
- Usa `timedelta` para somar ou subtrair tempo
- Usa `.strftime()` e `.strptime()` para converter entre texto e datas
- `.date()`, `.time()`, `.now()` e `.today()` são funções comuns

## Capítulo 8 – Gestão de Ficheiros

Guardar e ler dados de ficheiros é essencial para quase todos os programas úteis. Seja um simples `.txt`, um `.csv` com dados ou um `.json` com estrutura hierárquica, o Python torna tudo simples e elegante.

---

### Abrir ficheiros com `open()`

A função `open()` permite abrir um ficheiro para leitura ou escrita:

```
ficheiro = open("dados.txt", "r") # "r" = read (ler)
conteudo = ficheiro.read()
ficheiro.close()
```

 Mas atenção: **nunca te esqueças de fechar o ficheiro!**

---

### A forma recomendada: `with`

Com `with`, o ficheiro fecha-se automaticamente:

```
with open("dados.txt", "r") as f:
 conteudo = f.read()
 print(conteudo)
```

---

### Modos de abertura

| Modo | Significado                     |
|------|---------------------------------|
| "r"  | leitura                         |
| "w"  | escrita (apaga tudo)            |
| "a"  | acrescentar (append)            |
| "x"  | criar novo (erro se já existir) |
| "b"  | modo binário                    |

 "w" e "a" criam o ficheiro se ele não existir.

---

### Escrever em ficheiros

```
with open("mensagem.txt", "w") as f:
 f.write("Olá, mundo!\n")
 f.write("Segunda linha.")
```

---

## Ler linha a linha

```
with open("dados.txt", "r") as f:
 for linha in f:
 print(linha.strip())
```

`.readline()` vs `.readlines()`

```
linha1 = f.readline() # só uma linha
todas = f.readlines() # lista com todas as linhas
```

---

## Trabalhar com `.csv`

```
import csv
```

```
with open("alunos.csv", newline="") as f:
 leitor = csv.reader(f)
 for linha in leitor:
 print(linha)
```

### Escrever CSV:

```
with open("alunos.csv", "w", newline="") as f:
 escritor = csv.writer(f)
 escritor.writerow(["nome", "nota"])
 escritor.writerow(["Ana", 18])
```

---

## Trabalhar com `.json`

```
import json
```

# Ler JSON

```
with open("dados.json", "r") as f:
 dados = json.load(f)
 print(dados)
```

# Escrever JSON

```
info = {"nome": "Bruno", "idade": 25}
```

```
with open("dados.json", "w") as f:
 json.dump(info, f)
```

---

## Caminhos e diretórios

Usa `os` ou `pathlib` para trabalhar com pastas e caminhos:

```
import os
```

```
print(os.getcwd()) # diretório atual
```

```
os.chdir("subpasta") # mudar de pasta
os.listdir() # listar ficheiros
```

Com pathlib (mais moderno):

```
from pathlib import Path

caminho = Path("dados.txt")
if caminho.exists():
 print("O ficheiro existe!")
```

---

## Mini-desafios

1. Cria um ficheiro `.txt` com nome e idade inseridos pelo utilizador.
2. Lê um ficheiro `.txt` e conta quantas linhas contém.
3. Guarda uma lista de dicionários num ficheiro `.json`.
4. Cria um `.csv` com colunas “Produto” e “Preço”, escreve 3 linhas e depois lê-as.

---

## Resumo do capítulo

- Usa `with open(...)` para ler ou escrever ficheiros com segurança
- Usa os modos `"r"`, `"w"`, `"a"` conforme o objetivo
- Usa os módulos `csv`, `json`, `os` e `pathlib` para formatos e caminhos
- Aprende a trabalhar com texto, listas e dicionários em disco



## Capítulo 9 – Erros, Exceções e Depuração

Todos cometemos erros — até os programas! 🤪

Neste capítulo, vais aprender a **lidar com erros** de forma elegante, tornando o teu código mais **robusto, seguro** e **fácil de depurar**.

---

### Tipos de erros em Python

#### ✖ Erros de sintaxe (SyntaxError)

Acontecem quando esqueces dois pontos, parêntesis, etc.

```
if x > 5
 print("Erro de sintaxe!") # falta o ":"
```

#### ✨ Erros de execução (RuntimeError)

O programa começa bem, mas **explode a meio**:

```
x = int("abc") # ValueError
print(10 / 0) # ZeroDivisionError
```

#### 🌀 Erros lógicos

O código corre sem erros, mas o **resultado está errado**.

```
media = soma * 2 # quando querias dividir
```

---

### Capturar exceções com try / except

```
try:
 x = int(input("Número: "))
 print(10 / x)
except ValueError:
 print("Erro: valor não é número.")
except ZeroDivisionError:
 print("Erro: divisão por zero.")
```

---

### Usar else e finally

```
try:
 x = int(input("Idade: "))
except:
 print("Algo correu mal.")
else:
 print("Tudo correu bem.")
```

```
finally:
 print("Obrigado por usar o programa.")
```

| Bloco   | Executa se...               |
|---------|-----------------------------|
| try     | sempre                      |
| except  | houver erro                 |
| else    | não houver erro             |
| finally | sempre (ocorra ou não erro) |

---

## Lançar erros com `raise`

```
def validar_idade(idade):
 if idade < 0:
 raise ValueError("Idade não pode ser negativa.")

validar_idade(-5) # ValueError
```

---

## Depuração simples

Usar `print()` para inspecionar:

```
print("Antes do cálculo:", x)
```

Usar breakpoints no VS Code 

1. Clica à esquerda da linha para marcar um ponto vermelho.
2. Carrega em F5 para correr o script no modo de depuração.
3. Inspecciona variáveis passo a passo.

---

## Boas práticas de tratamento de erros

- ✓ Sê específico no `except` (ex: `ValueError`)
- ✓ Usa mensagens de erro informativas
- ✓ Não abuses de `try/except` para lógica normal
- ✓ Usa `finally` para libertar recursos (ex: ficheiros, conexões)

---

## Mini-desafios

1. Pede um número ao utilizador e divide 100 por esse número. Trata os erros possíveis.
  2. Cria uma função `pedir_idade()` que só aceita valores inteiros positivos.
  3. Lê um ficheiro `.txt`, tratando o erro se ele não existir.
  4. Usa `raise` para impedir que alguém insira um nome vazio num formulário.
-

## Resumo do capítulo

- Erros acontecem — trata-os com `try`, `except`, `else`, `finally`
- Usa `raise` para criar os teus próprios erros
- Aprende a depurar com `print()` e breakpoints
- Torna o teu código seguro e previsível!

## Capítulo 10 – Programação Orientada por Objetos (POO)

Até agora, aprendeste a escrever programas em Python de forma **estrutural** — com funções, variáveis e estruturas de dados. Mas em sistemas maiores, é importante organizar o código de forma mais **modular**, **reutilizável** e **lógica**.

A Programação Orientada por Objetos (POO) permite criar **modelos do mundo real** usando objetos com propriedades e comportamentos.

---

### O que é um objeto?

Um **objeto** é uma estrutura que representa algo com:

- **Atributos** (dados) → Ex: nome, idade
- **Métodos** (ações) → Ex: falar(), dormir()

 Um objeto é uma **instância de uma classe**.

---

### Definir uma classe com `class`

```
class Pessoa:
 def __init__(self, nome, idade):
 self.nome = nome
 self.idade = idade

 def apresentar(self):
 print(f"Olá, eu sou o/a {self.nome} e tenho {self.idade} anos.")
```

#### Explicação:

- `class Pessoa`: define a classe
  - `__init__`: método especial de inicialização
  - `self`: refere-se ao próprio objeto
  - `self.nome`: cria um atributo no objeto
- 

### Criar e usar objetos

```
joao = Pessoa("João", 30)
ana = Pessoa("Ana", 25)
```

```
joao.apresentar()
ana.apresentar()
```

---

## Atributos e métodos

### Atributos

```
print(joao.nome) # João
joao.idade += 1 # Faz anos!
```

### Métodos

```
joao.apresentar()
```

---

## Encapsulamento

Controlar o acesso aos dados:

```
class Conta:
 def __init__(self, saldo):
 self.__saldo = saldo # privado

 def ver_saldo(self):
 return self.__saldo

 def depositar(self, valor):
 if valor > 0:
 self.__saldo += valor
```

---

## Herança

Permite criar classes **filhas** que herdam de outra classe:

```
class Aluno(Pessoa):
 def __init__(self, nome, idade, curso):
 super().__init__(nome, idade)
 self.curso = curso

 def apresentar(self):
 super().apresentar()
 print(f"Estudo {self.curso}.")
```

---

## Composição

Uma classe **usa** outra, mas **não herda** dela:

```
class Endereco:
 def __init__(self, cidade):
 self.cidade = cidade
```

```
class Empresa:
```

```
def __init__(self, nome, endereco):
 self.nome = nome
 self.endereco = endereco

morada = Endereco("Lisboa")
empresa = Empresa("TechX", morada)
print(empresa.endereco.cidade) # Lisboa
```

---

## Métodos especiais (\_\_str\_\_, \_\_repr\_\_)

```
class Produto:
 def __init__(self, nome, preco):
 self.nome = nome
 self.preco = preco

 def __str__(self):
 return f"{self.nome} - €{self.preco:.2f}"

p = Produto("Livro", 12.5)
print(p) # Chama __str__ automaticamente
```

---

## Operadores personalizados

```
def __lt__(self, outro):
 return self.preco < outro.preco # menor que
```

---

## Mini-projeto

Cria uma classe Livro com:

- Atributos: título, autor, ano
- Método detalhes() que imprime tudo

Depois, cria uma classe Biblioteca que guarda uma lista de livros:

- adicionar\_livro()
  - listar\_livros()
- 

## Resumo do capítulo

- class: define uma **estrutura personalizada**
- Objetos têm **atributos e métodos**
- Usa `__init__`, `self`, e métodos como `__str__`
- Usa **herança** para reaproveitar código
- Usa **composição** para construir relações

- POO organiza o código para ser mais **modular e reutilizável**

## Revisão rápida: O essencial de uma classe

```
class Pessoa:
 def __init__(self, nome, idade):
 self.nome = nome
 self.idade = idade

 def apresentar(self):
 print(f"Olá, eu sou o/a {self.nome} e tenho {self.idade} anos.")
```

---

### ◇ A diferença entre `__str__` e `__repr__`

- `__str__`: representação **amigável** para o utilizador (ex: quando fazemos `print(objeto)`)
- `__repr__`: representação **oficial** do objeto — usada para debugging e quando se imprime uma lista de objetos

#### Exemplo:

```
class Produto:
 def __init__(self, nome, preco):
 self.nome = nome
 self.preco = preco

 def __str__(self):
 return f"{self.nome} - €{self.preco:.2f}"

 def __repr__(self):
 return f"Produto(nome='{self.nome}', preco={self.preco})"

p = Produto("Caderno", 3.5)
print(p) # Caderno - €3.50
print([p]) # [Produto(nome='Caderno', preco=3.5)]
```

---

### ◇ Comparar objetos com operadores (`__eq__`, `__lt__`, etc.)

```
class Livro:
 def __init__(self, titulo, paginas):
 self.titulo = titulo
 self.paginas = paginas

 def __eq__(self, outro):
 return self.paginas == outro.paginas

 def __lt__(self, outro):
 return self.paginas < outro.paginas
```

```
livro1 = Livro("Python 101", 250)
livro2 = Livro("Python Avançado", 250)

print(livro1 == livro2) # True
print(livro1 < livro2) # False
```

---

### ◇ Atributos e métodos de classe (@classmethod)

- Atributos de instância: pertencem a cada objeto
- Atributos de classe: comuns a **todas** as instâncias

```
class Conta:
 taxa_juros = 0.02 # atributo de classe

 def __init__(self, saldo):
 self.saldo = saldo

 @classmethod
 def alterar_taxa(cls, nova_taxa):
 cls.taxa_juros = nova_taxa
```

---

### ◇ Métodos estáticos (@staticmethod)

Usados quando o método **não precisa de self nem cls**:

```
class Matematica:
 @staticmethod
 def quadrado(x):
 return x * x

print(Matematica.quadrado(5)) # 25
```

---

### ◇ Encapsulamento real com @property

Em vez de usar `get_` e `set_`, podes criar **acesso controlado elegante**:

```
class Pessoa:
 def __init__(self, nome):
 self._nome = nome # privado por convenção

 @property
 def nome(self):
 return self._nome

 @nome.setter
 def nome(self, valor):
 if len(valor) < 3:
```



```
 raise ValueError("Nome muito curto.")
 self._nome = valor

p = Pessoa("Ana")
print(p.nome) # usa o getter
p.nome = "Joana" # usa o setter
```

---

## ◇ Classe abstrata e polimorfismo 🧬

```
from abc import ABC, abstractmethod

class Animal(ABC):
 @abstractmethod
 def emitir_som(self):
 pass

class Gato(Animal):
 def emitir_som(self):
 return "Miau!"

class Cão(Animal):
 def emitir_som(self):
 return "Au au!"

animais = [Gato(), Cão()]
for a in animais:
 print(a.emitir_som()) # Miau! / Au au!
```

---

## 🧠 Boas práticas em POO com Python

- ✓ Usa nomes claros e descritivos
  - ✓ Começa com `__init__`, depois evolui para `@property`, `__str__`, etc.
  - ✓ Prefere composição a herança quando possível
  - ✓ Evita tornar atributos verdadeiramente privados, usa `_` como convenção
  - ✓ Testa com `isinstance()` e `type()` para robustez
- 

## 🔧 Desafios Avançados

1. Cria uma classe `Funcionario` com salário base e método `calcular_salario()`. Depois cria uma subclasse `Comercial` com comissões.
2. Cria uma classe `Temperatura` com atributo em Celsius e propriedades para converter automaticamente para Fahrenheit e Kelvin.
3. Cria uma classe `Poligono` abstrata e classes `Triangulo` e `Quadrado` com métodos `area()` e `perimetro()` implementados.

## Capítulo 11 – Introdução a Bibliotecas Científicas

Python brilha no mundo acadêmico e científico graças às suas bibliotecas especializadas, que permitem realizar desde simples cálculos até análises estatísticas complexas.

Neste capítulo, vais explorar cinco bibliotecas essenciais para estudantes e investigadores:

- `math`: matemática básica e funções avançadas
- `statistics`: estatística descritiva
- `random`: simulações e geração aleatória
- `decimal`: cálculos com alta precisão decimal
- `fractions`: frações matemáticas reais (com denominadores)

---

### `math`: funções matemáticas básicas e avançadas

```
import math
```

| Função                         | Descrição                |
|--------------------------------|--------------------------|
| <code>math.sqrt(x)</code>      | raiz quadrada            |
| <code>math.pow(x, y)</code>    | potência (x elevado a y) |
| <code>math.sin(x)</code>       | seno (em radianos!)      |
| <code>math.pi</code>           | constante $\pi$          |
| <code>math.log(x)</code>       | logaritmo natural        |
| <code>math.factorial(n)</code> | fatorial de n            |

```
print(math.sqrt(25)) # 5.0
print(math.pi) # 3.141592...
print(math.sin(math.pi / 2)) # 1.0
```

---

### `statistics`: estatística descritiva

```
import statistics as stats
```

```
dados = [10, 15, 20, 20, 30]
print(stats.mean(dados)) # média
print(stats.median(dados)) # mediana
print(stats.mode(dados)) # moda
print(stats.stdev(dados)) # desvio padrão
```

 Ideal para análise de resultados, estudos de mercado ou experimentos científicos.

---

### `random`: geração de números aleatórios

```
import random
```

| Função | Exemplo |
|--------|---------|
|--------|---------|

| Função                             | Exemplo                |
|------------------------------------|------------------------|
| <code>random.random()</code>       | número entre 0 e 1     |
| <code>random.randint(a, b)</code>  | inteiro entre a e b    |
| <code>random.choice(lista)</code>  | escolhe aleatoriamente |
| <code>random.shuffle(lista)</code> | embaralha a lista      |

```
print(random.randint(1, 6)) # simula um dado
print(random.choice(["cara", "coroa"]))
```

 Muito usado em simulações de Monte Carlo, jogos, sorteios, testes automatizados.

### decimal: precisão decimal elevada

```
from decimal import Decimal, getcontext
```

```
getcontext().prec = 6 # número de casas decimais
x = Decimal("10.12345")
y = Decimal("3.00000")
print(x / y)
```

 Ideal para finanças, contabilidade ou quando a precisão é crítica.

### fractions: trabalhar com frações reais

```
from fractions import Fraction
```

```
f1 = Fraction(1, 3)
f2 = Fraction(1, 6)
print(f1 + f2) # 1/2
```

 Útil para ensino de matemática, álgebra simbólica e situações em que a aproximação decimal não é desejada.

### Mini-desafios

1. Gera 100 números aleatórios entre 1 e 6 e calcula a média.
2. Cria uma função que receba 3 notas e calcule a média, mediana e desvio padrão.
3. Constrói uma simulação simples de moeda ao ar (100 lançamentos) e mostra quantas vezes saiu "cara".
4. Converte 5 operações financeiras com `Decimal` para garantir precisão.
5. Usa `Fraction` para somar  $1/4 + 1/3$  e apresenta o resultado simplificado.

### Resumo do capítulo

- `math`: funções matemáticas tradicionais (seno, raiz, pi)

- `statistics`: média, moda, mediana, desvio padrão
- `random`: simulações e sorteios
- `decimal`: operações com precisão controlada
- `fractions`: manipulação exata de frações

## Capítulo 12 – NumPy e Computação Numérica ⚡ 📐

O NumPy (Numerical Python) é a base da computação científica em Python. Permite realizar cálculos com **arrays** (vetores e matrizes) de forma muito mais rápida e eficiente que as listas tradicionais.

Neste capítulo vais aprender:

- O que é um array NumPy
- Como criar e manipular arrays
- Operações vetoriais e matriciais
- Estatísticas e álgebra linear com alta performance

---

### Por que usar NumPy? 🚀

|                  |                    |
|------------------|--------------------|
| Listas em Python | Arrays NumPy       |
| Flexíveis        | Muito mais rápidos |
| Misturam tipos   | Tipagem forte      |
| Não otimizadas   | Otimizadas em C    |

NumPy é usado por bibliotecas como Pandas, SciPy, Scikit-Learn e TensorFlow.

---

### Começar com NumPy 📦

```
pip install numpy
```

```
import numpy as np
```

---

### Criar arrays (vetores e matrizes) 📊

```
vetor = np.array([1, 2, 3])
matriz = np.array([[1, 2], [3, 4]])
```

Gerar arrays rapidamente:

```
np.zeros(4) # [0. 0. 0. 0.]
np.ones((2, 3)) # matriz 2x3 cheia de 1
np.arange(0, 10, 2) # [0 2 4 6 8]
np.linspace(0, 1, 5) # 5 valores entre 0 e 1
```

---

### Operações com arrays + - × ÷

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b) # [11 22 33]
```

```
print(a * 2) # [2 4 6]
print(a ** 2) # [1 4 9]
```

Tudo é **vetorizado** — não precisas de ciclos for.

---

## Indexação e slicing

```
a = np.array([10, 20, 30, 40])
print(a[1]) # 20
print(a[1:3]) # [20 30]
```

---

## Estatísticas com NumPy

```
dados = np.array([2, 4, 6, 8])

print(dados.mean()) # média
print(dados.std()) # desvio padrão
print(dados.var()) # variância
print(dados.min(), dados.max()) # mínimo e máximo
```

---

## Operações matriciais

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 2]])

print(A + B)
print(A @ B) # multiplicação de matrizes
print(np.dot(A, B)) # equivalente
```

---

## Álgebra linear com `np.linalg`

```
import numpy.linalg as la

M = np.array([[2, 1], [1, 3]])

print(la.det(M)) # determinante
print(la.inv(M)) # inversa
print(la.eig(M)) # autovalores e autovetores
```

---

## Comparação com listas

```
import time

lista = list(range(1000000))
```

```
inicio = time.time()
[x**2 for x in lista]
print("Lista:", time.time() - inicio)
```

```
array = np.arange(1000000)
inicio = time.time()
array ** 2
print("NumPy:", time.time() - inicio)
```

 NumPy é **muito mais rápido**, porque usa código compilado (C).

---

### Mini-desafios

1. Cria um array de 1000 valores aleatórios e calcula média, mínimo, máximo e desvio.
  2. Cria duas matrizes  $3 \times 3$  e faz a multiplicação entre elas.
  3. Usa `np.linspace` para criar 20 valores entre  $-2\pi$  e  $2\pi$ , e calcula o seno de cada um.
  4. Resolve um sistema de equações lineares usando `np.linalg.solve`.
- 

### Resumo do capítulo

- NumPy é a base da computação numérica com arrays otimizados
- Arrays permitem operações **vetorizadas** e rápidas
- Facilita cálculo estatístico, álgebra linear e manipulação de dados científicos

## Capítulo 13 – Pandas para Análise de Dados

O Pandas é a ferramenta principal para análise de dados em Python. Permite-te trabalhar com tabelas (como folhas de Excel), fazer filtros, agrupar, limpar dados e importar/exportar ficheiros de forma simples e poderosa.

---

### Começar com Pandas

```
pip install pandas

import pandas as pd
```

---

### Séries: listas com índice

```
notas = pd.Series([14, 16, 12, 18])
print(notas)
```

Com rótulos personalizados:

```
alunos = pd.Series([14, 16, 12], index=["Ana", "Bruno", "Carla"])
print(alunos["Bruno"]) # 16
```

---

### DataFrames: tabelas de dados

```
dados = {
 "Nome": ["Ana", "Bruno", "Carla"],
 "Nota": [14, 16, 12],
 "Curso": ["Bio", "Eng", "Mat"]
}

df = pd.DataFrame(dados)

print(df.head()) # primeiras linhas
print(df.info()) # resumo
print(df.describe()) # estatísticas básicas
```

---

### Acessar colunas e linhas

```
print(df["Nota"])
print(df.Nome)

print(df.loc[1]) # por índice
print(df.iloc[0:2]) # slicing
```

---



## Filtrar dados

```
print(df[df["Nota"] >= 14])

print(df[(df["Nota"] >= 14) & (df["Curso"] == "Eng")])
```

---

## Adicionar e remover colunas

```
df["Aprovado"] = df["Nota"] >= 10
df.drop("Curso", axis=1, inplace=True)
```

---

## Agrupar e resumir dados

```
print(df.groupby("Aprovado")["Nota"].mean())
```

---

## Ordenar valores

```
df.sort_values("Nota", ascending=False, inplace=True)
```

---

## Lidar com dados em falta

```
df.isnull() # verifica
df.dropna() # remove
df.fillna(0) # substitui
```

---

## Importar e exportar ficheiros

### CSV

```
df = pd.read_csv("dados.csv")
df.to_csv("resultado.csv", index=False)
```

### Excel

```
df = pd.read_excel("dados.xlsx")
df.to_excel("resultado.xlsx", index=False)
```

---

## Mini-desafios

1. Cria um DataFrame com nome, nota e curso de 5 alunos. Filtra só os aprovados.
2. Lê um ficheiro .csv com dados de produtos e mostra os 3 mais caros.
3. Adiciona uma coluna com “IVA incluído” num DataFrame de preços.
4. Agrupa dados por categoria e mostra a média de cada grupo.

5. Lê um ficheiro Excel e salva apenas as colunas desejadas num novo CSV.
- 

## Resumo do capítulo

- Pandas é a biblioteca principal para dados tabulares
- `Series` → colunas individuais com índice
- `DataFrame` → tabelas completas com colunas e linhas
- Permite importar/exportar ficheiros, filtrar, agrupar e limpar dados facilmente

# Capítulo 14 – Visualização de Dados com Matplotlib e Seaborn



“Uma imagem vale por mil linhas de código.”

Neste capítulo, vais aprender a criar **gráficos claros e apelativos** com as duas bibliotecas mais populares para visualização de dados em Python:

- Matplotlib: flexível e poderosa
- Seaborn: elegante e orientada a dados

---

## Começar com as bibliotecas

```
pip install matplotlib seaborn
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

---

## Gráficos com Matplotlib

### Gráfico de linhas

```
import matplotlib.pyplot as plt
```

```
anos = [2020, 2021, 2022, 2023]
vendas = [150, 200, 180, 220]
```

```
plt.plot(anos, vendas)
plt.title("Vendas por Ano")
plt.xlabel("Ano")
plt.ylabel("Milhares €")
plt.grid(True)
plt.show()
```

### Gráfico de barras

```
produtos = ["A", "B", "C"]
quantidades = [120, 90, 150]
```

```
plt.bar(produtos, quantidades, color="lightblue")
plt.title("Vendas por Produto")
plt.show()
```

### Histograma

```
idades = [22, 25, 21, 30, 28, 25, 31, 22, 40]
```

```
plt.hist(idades, bins=5, color="orange", edgecolor="black")
plt.title("Distribuição de Idades")
plt.xlabel("Idade")
```

```
plt.ylabel("Frequência")
plt.show()
```

---

## Seaborn: gráficos bonitos com pouco código

```
import seaborn as sns
import pandas as pd
```

```
Exemplo com DataFrame
df = pd.DataFrame({
 "idade": [22, 25, 30, 35, 40],
 "salario": [1000, 1300, 1500, 1700, 2000],
 "departamento": ["RH", "TI", "TI", "RH", "Finanças"]
})
```

### Dispersão (scatterplot)

```
sns.scatterplot(data=df, x="idade", y="salario", hue="departamento")
plt.title("Idade vs Salário")
plt.show()
```

### Boxplot (distribuição e outliers)

```
sns.boxplot(data=df, x="departamento", y="salario")
plt.title("Salário por Departamento")
plt.show()
```

### Heatmap (mapa de calor de correlações)

```
correlacoes = df.corr(numeric_only=True)
sns.heatmap(correlacoes, annot=True, cmap="Blues")
plt.title("Correlação entre Variáveis")
plt.show()
```

---

## Estilo e estética

```
sns.set(style="whitegrid") # tema
plt.figure(figsize=(8, 5)) # tamanho do gráfico
```

---

## Guardar o gráfico como imagem

```
plt.savefig("grafico.png", dpi=300)
```

---

## Mini-desafios

1. Cria um gráfico de barras com os gastos mensais do teu orçamento.
2. Representa a evolução da temperatura ao longo de uma semana com um line plot.
3. Usa Seaborn para comparar salários entre áreas com boxplot.

4. Gera um heatmap de correlação entre colunas numéricas de um `.csv`.
  5. Guarda o gráfico num ficheiro `.png`.
- 

## Resumo do capítulo

- `Matplotlib` → flexível para gráficos de linhas, barras, histogramas
- `Seaborn` → ideal para visualizações rápidas com `DataFrames`
- Gráficos ajudam a explorar, comunicar e interpretar dados
- Personaliza estilo, cor, título e exporta imagens

## Capítulo 15 – APIs e Web Scraping

Hoje em dia, grande parte dos dados disponíveis online pode ser acedida através de **APIs** (interfaces de programação) ou recolhida diretamente de páginas web com **web scraping**.

Neste capítulo vais aprender:

- O que é uma API e como interagir com ela com requests
  - Como fazer scraping de páginas HTML com BeautifulSoup
  - Como tratar e transformar os dados recolhidos
- 

### O que é uma API?

Uma **API (Application Programming Interface)** permite que o teu programa **fale com outro sistema online** para receber ou enviar dados estruturados, geralmente em **JSON**.

Exemplos:

- Previsão do tempo
  - Dados de ações, moedas, futebol
  - Tradução automática
- 

### Usar a biblioteca requests

```
pip install requests
```

#### Fazer um pedido GET

```
import requests
```

```
url = "https://api.exemplo.com/dados"
resposta = requests.get(url)
```

```
print(resposta.status_code) # 200 = OK
print(resposta.json()) # converte JSON em dicionário
```

---

### Exemplo com API pública: exchangerate.host

```
import requests
```

```
res = requests.get("https://api.exchangerate.host/latest?base=EUR")
dados = res.json()
```

```
print(dados["rates"]["USD"]) # taxa EUR → USD
```

---

## Enviar parâmetros na URL

```
params = {
 "base": "USD",
 "symbols": "EUR,GBP"
}
```

```
res = requests.get("https://api.exchangerate.host/latest", params=params)
print(res.json())
```

---

## Introdução ao Web Scraping

Quando não existe API, podes **extrair dados diretamente do HTML da página**. Para isso usas:

```
pip install beautifulsoup4

from bs4 import BeautifulSoup
import requests
```

---

## Exemplo: extrair títulos de uma página

```
url = "https://www.nytimes.com"
html = requests.get(url).text
soup = BeautifulSoup(html, "html.parser")

titulos = soup.find_all("h3")

for t in titulos[:5]:
 print(t.get_text())
```

---

## Métodos úteis do BeautifulSoup

| Método             | Significado                         |
|--------------------|-------------------------------------|
| find()             | devolve o primeiro elemento         |
| find_all()         | devolve todos os elementos          |
| .get_text()        | extrai o texto interno              |
| .attrs ou ["href"] | accede a atributos HTML (ex: links) |

---

## Cuidados e ética no scraping

- Verifica se o site permite scraping (robots.txt)
- Não faças muitos pedidos por segundo
- Usa headers com user-agent para simular navegador

```
headers = {"User-Agent": "Mozilla/5.0"}
res = requests.get(url, headers=headers)
```

---

## Mini-desafios

1. Usa a API <https://api.agify.io/?name=joao> para prever idades com base em nomes.
  2. Extraí os títulos principais de uma página de notícias.
  3. Acede a um site de câmbio e obtém a taxa EUR→USD.
  4. Cria uma função que recebe um nome e usa <https://api.genderize.io> para prever o género.
  5. Faz scraping da Wikipédia e extraí os nomes dos países da Europa.
- 

## Resumo do capítulo

- APIs permitem aceder a dados online de forma estruturada (JSON)
- Usa `requests` para enviar pedidos e receber respostas
- Usa `BeautifulSoup` para extrair texto e atributos de HTML
- Web scraping é útil quando **não há API**, mas deve ser feito com **respeito pelas regras dos sites**



## Capítulo 16 – Ambientes Virtuais, Módulos e Pacotes

À medida que os teus projetos Python crescem, vais querer:

- Organizar bem o teu código (em **módulos e pacotes**);
- Gerir dependências específicas para cada projeto (com **ambientes virtuais**);
- Evitar conflitos entre bibliotecas.

Este capítulo vai mostrar-te como manter tudo **isolado, organizado e profissional**.

---

### Módulos: separar para conquistar

Um **módulo** é simplesmente um ficheiro `.py` com funções, variáveis ou classes que podes reutilizar.

Exemplo:

`matematica.py`:

```
def somar(a, b):
 return a + b
```

```
def subtrair(a, b):
 return a - b
```

`programa.py`:

```
import matematica
```

```
print(matematica.somar(5, 3)) # 8
```

---

```
from ... import ... 
```

```
from matematica import somar
```

```
print(somar(2, 2))
```

---

### Pacotes: pastas de módulos

Um **pacote** é uma **pasta** com vários módulos e um ficheiro `__init__.py` (pode estar vazio).

```
meu_pacote/
├── __init__.py
├── contas.py
└── conversoes.py
```

Usar um pacote:

```
from meu_pacote.contas import somar
```

---

## Instalar bibliotecas com pip

Instalação global ou no ambiente atual:

```
pip install numpy
```

Atualizar:

```
pip install --upgrade pandas
```

Ver instaladas:

```
pip list
```

---

## Criar o teu próprio pacote

1. Cria uma pasta com o nome do pacote.
  2. Inclui os teus módulos e um `__init__.py`.
  3. (Opcional) Cria um `setup.py` para publicação futura.
- 

## Ambientes virtuais com venv

Permite-te **isolar bibliotecas por projeto**. Ideal para evitar conflitos entre projetos.

**Criar ambiente:**

```
python -m venv venv
```

**Ativar:**

- Windows:

```
venv\Scripts\activate
```

- macOS/Linux:

```
source venv/bin/activate
```

**Instalar dentro do ambiente:**

```
pip install pandas
```

**Desativar:**

```
deactivate
```

---

## Usar ficheiro requirements.txt

**Guardar bibliotecas:**

```
pip freeze > requirements.txt
```

Instalar de um projeto:

```
pip install -r requirements.txt
```

---

### Mini-desafios

1. Cria um módulo chamado `ferramentas.py` com duas funções úteis e importa-as noutra ficheiro.
  2. Organiza um projeto com 2 módulos diferentes dentro de um pacote.
  3. Cria um ambiente virtual, instala `requests` e verifica que só está disponível nesse projeto.
  4. Usa `pip freeze` para gerar um ficheiro `requirements.txt`.
  5. Publica um pacote simples no GitHub para partilhar com colegas.
- 

### Resumo do capítulo

- **Módulo** = ficheiro `.py` com funções/classes reutilizáveis
- **Pacote** = pasta com módulos + `__init__.py`
- Usa `pip` para instalar e gerir bibliotecas
- Usa `venv` para ambientes virtuais e evitar conflitos entre projetos
- Usa `requirements.txt` para guardar dependências do projeto

## Capítulo 17 – Automação e Tarefas Comuns

Um dos maiores superpoderes do Python é **automatizar tarefas** do dia a dia: organizar ficheiros, enviar emails, fazer backups, verificar preços ou até enviar alertas em tempo real.

Neste capítulo vais aprender a criar scripts simples mas poderosos para **poupar tempo e evitar erros humanos**.

---

### Automatizar renomeações de ficheiros

```
import os

pasta = "meus_ficheiros"

for i, nome in enumerate(os.listdir(pasta), start=1):
 novo_nome = f"ficheiro_{i}.txt"
 os.rename(os.path.join(pasta, nome), os.path.join(pasta, novo_nome))
```

---

### Enviar emails com Python

```
import smtplib
from email.mime.text import MIMEText

msg = MIMEText("Olá! Este é um email automático.")
msg["Subject"] = "Aviso automático"
msg["From"] = "teu_email@gmail.com"
msg["To"] = "destinatario@gmail.com"

with smtplib.SMTP_SSL("smtp.gmail.com", 465) as servidor:
 servidor.login("teu_email@gmail.com", "palavra_passe")
 servidor.send_message(msg)
```

⚠ Para Gmail, debes ativar a opção de “apps menos seguros” ou usar uma **App Password**.

---

### Criar alertas automáticos (ex: preço abaixo de X)

```
import requests

def verificar_preco():
 resposta = requests.get("https://api.exemplo.com/preco")
 preco = resposta.json()["preco"]
 if preco < 100:
 print("⚠ Alerta: Preço abaixo de 100!")

verificar_preco()
```

---

## Agendar tarefas com `schedule`

```
pip install schedule
```

```
import schedule
import time
```

```
def tarefa():
 print("Tarefa executada!")
```

```
schedule.every(10).seconds.do(tarefa)
schedule.every().day.at("09:00").do(tarefa)
```

```
while True:
 schedule.run_pending()
 time.sleep(1)
```

---

## Trabalhar com datas e backups

```
import shutil
import datetime
```

```
hoje = datetime.date.today().isoformat()
shutil.copy("base_de_dados.db", f"backup_{hoje}.db")
```

---

## Limpeza automática de pastas

```
import os
```

```
pasta = "temp"
```

```
for ficheiro in os.listdir(pasta):
 if ficheiro.endswith(".tmp"):
 os.remove(os.path.join(pasta, ficheiro))
```

---

## Monitorizar uma pasta (ex: nova imagem)

```
import os
import time
```

```
pasta = "fotos"
antes = set(os.listdir(pasta))
```

```
while True:
 agora = set(os.listdir(pasta))
 novos = agora - antes
 if novos:
```

```
print("Novos ficheiros:", novos)
antes = agora
time.sleep(5)
```

---

## Mini-desafios

1. Cria um script que organiza ficheiros numa pasta por extensão (.txt, .jpg, etc.).
  2. Envia um email automático a ti próprio com um relatório de notas ou tarefas.
  3. Agenda um alerta diário com schedule para estudar Python às 18h.
  4. Cria backups automáticos de um ficheiro importante com data no nome.
  5. Faz um script que apaga ficheiros com mais de 7 dias numa pasta temporária.
- 

## Resumo do capítulo

- Python pode **automatizar tarefas repetitivas** com poucas linhas
- Usa bibliotecas como os, shutil, smtplib, schedule e datetime
- Agendar, renomear, copiar, apagar e notificar são tarefas simples com Python

## Capítulo 18 – Projetos Finais para Consolidar

Agora que dominas os fundamentos do Python, chegou a altura de **pôr tudo em prática**. Neste capítulo vais explorar 3 projetos aplicados a contextos reais — ideais para alunos universitários, investigadores ou entusiastas da automação e ciência de dados.

Cada projeto pode ser adaptado ao teu nível e expandido conforme a tua criatividade e área de interesse.

---

### Projeto 1 – Análise de Dados Económicos (Pandas + Matplotlib)

**Objetivo:** importar dados económicos (ex: PIB, desemprego), fazer limpeza, análise estatística e gerar visualizações.

#### Funcionalidades:

- Importar ficheiro CSV (ex: `indicadores.csv`)
- Limpar valores em falta
- Calcular médias, tendências, correlações
- Visualizar com gráficos de linha e barras

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("indicadores.csv")
df.dropna(inplace=True)

df["PIB"].plot(title="PIB ao longo do tempo")
plt.show()

print(df.groupby("País")["Desemprego"].mean())
```

---

### Projeto 2 – Simulador de Carteira de Investimentos (NumPy + POO)

**Objetivo:** modelar uma carteira com ativos financeiros, simular rendimentos, calcular retornos esperados e risco.

#### Funcionalidades:

- Classe Ativo: nome, retorno esperado, risco (desvio)
- Classe Carteira: lista de ativos e respetivos pesos
- Simulação de retorno médio com `np.random.normal`
- Cálculo do retorno médio ponderado e risco total

```
class Ativo:
 def __init__(self, nome, retorno, desvio):
 self.nome = nome
 self.retorno = retorno
 self.desvio = desvio
```

```
def simular(self):
 return np.random.normal(self.retorno, self.desvio)

depois cria-se a classe Carteira com métodos para calcular retorno total
```

---

## Projeto 3 – Ferramenta de Scraping de Dados Públicos (Requests + JSON)

**Objetivo:** aceder a dados públicos de uma API e apresentar visualmente ou guardar num ficheiro.

**Exemplo:** previsão meteorológica

```
import requests

def previsao(cidade):
 url = f"https://api.open-meteo.com/v1/forecast?latitude=38.7&longitude=-
9.1&hourly=temperature_2m"
 r = requests.get(url)
 dados = r.json()
 print(dados["hourly"]["temperature_2m"][:5])

previsao("Lisboa")
```

**Possíveis variações:**

- Extrair preços de livros ou produtos
  - Extrair dados de futebol ou desporto
  - Guardar ficheiro .json ou .csv com os dados recolhidos
- 

## Sugestões de Projetos Alternativos

- Calculadora científica interativa com GUI (Tkinter ou Streamlit)
  - Gerador de passwords seguras
  - Analisador de texto (frequência de palavras, contagem de frases)
  - Script que verifica automaticamente se um site está online
- 

## Resumo do capítulo

- Os projetos aplicam o que aprendeste: estruturas, funções, ficheiros, POO, bibliotecas
  - Usa bibliotecas como pandas, numpy, matplotlib, requests
  - Automatiza, analisa e visualiza dados de forma simples e eficaz
  - São o ponto de partida ideal para portefólios, relatórios, dissertações ou apresentações
-